

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Алгоритми та методи обчислень

Лабораторна робота

№1

«Поняття алгоритму. Задавання алгоритмів у вигляді блок-схем».

Виконав: студент
групи ІО-42
Куліков М. М.
Номер у списку групи: 9
Перевірив:
Порєв В. М.

Лабораторна робота №1

Тема: «Поняття алгоритму. Задавання алгоритмів у вигляді блок-схем».

Мета: Навчитися створювати блок-схеми лінійного алгоритму; розгалуженого алгоритму та циклічного алгоритму за допомогою редактора блок-схем afse або іншого довільного редактора.

Загальне завдання:

1. Відповідно до варіанту завдання розробити блок-схеми обчислення виразів для лінійного алгоритму, алгоритму, що розгалужується та циклічного алгоритму. У відповідності до блок-схеми створити програму обчислення виразу алгоритмічною мовою, узгодженою з викладачем.

Вимоги до програмного забезпечення:

- Модульна структура програми;
- Уведення даних із клавіатури та з зовнішнього файлу;
- Перевірка коректності введених даних;
- Інтерфейс у стилі GUI.

Індивідуальне завдання:

Завдання виконане згідно 9 варіанту.

Таблиця 1 – Завдання згідно варіанту

№ вар	Лінійний	Що розгалужується	Циклічний
9	$Y1 = \sqrt[3]{5 + c \cdot \sqrt{b + 5\sqrt{a}}}$	Обчислити, якщо $k > 10$ $y = \sqrt{k \cdot \sqrt{d^2} + d \cdot \sqrt{k^2}}$ інакше $y = (k + d)^2$	Обчислити $f = \prod_{i=1}^n (a_i + b_{i+1}) + \sum_i^n (a_{i+1} \cdot b_i)$

Блок-схеми алгоритмів:

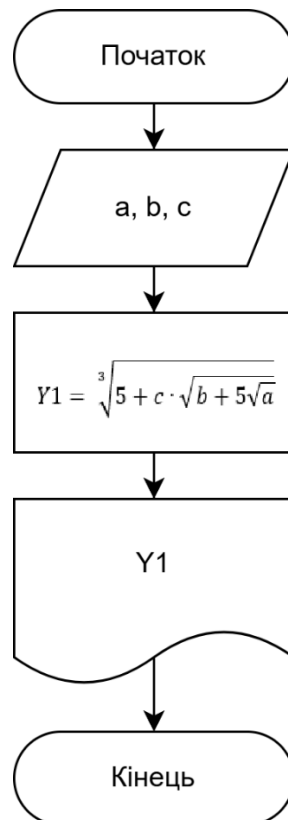


Рисунок 1 – Блок схема лінійного алгоритму

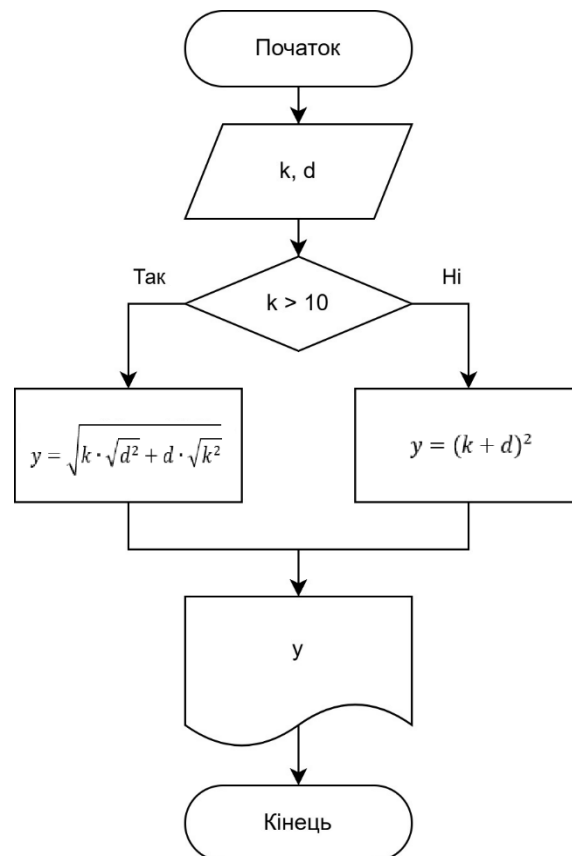


Рисунок 2 – Блок схема алгоритму, що розгалужується

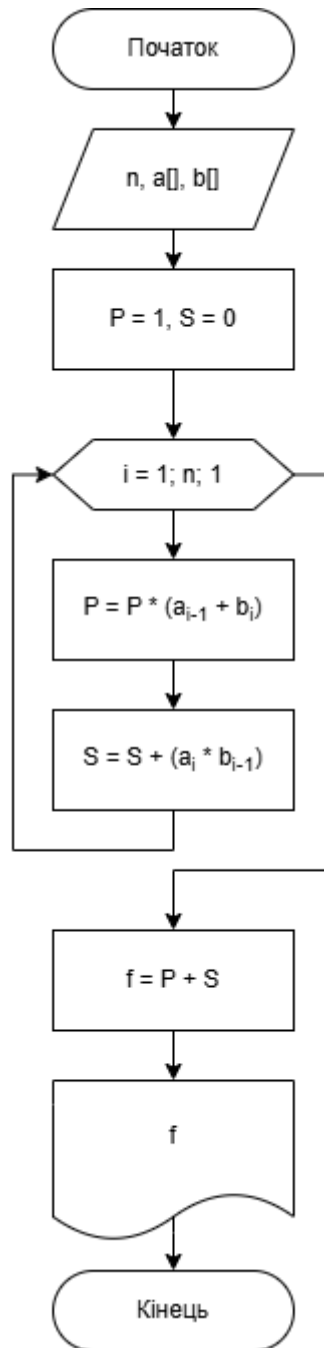


Рисунок 3 – Блок схема циклічного алгоритму

Код програми:

Вміст файл LR1.py:

```

import tkinter as tk
from tkinter import ttk, messagebox, filedialog
from PIL import Image, ImageTk
import random
import os
from MathLogic import CalculatorLogic

```

```

class MainWindow(tk.Tk):
    def __init__(self):
        super().__init__()

```

```

self.title("JP1")

menubar = tk.Menu(self)
file_menu = tk.Menu(menubar, tearoff=0)
file_menu.add_command(label="Зберегти", command=self.save_file)
file_menu.add_command(label="Відкрити", command=self.load_file)
menubar.add_cascade(label="Файл", menu=file_menu)
self.config(menu=menubar)

self.notebook = ttk.Notebook(self)
self.notebook.pack(expand=True, fill="both", padx=10, pady=10)

self.tab1 = ttk.Frame(self.notebook)
self.tab2 = ttk.Frame(self.notebook)
self.tab3 = ttk.Frame(self.notebook)

self.notebook.add(self.tab1, text="Лінійний")
self.notebook.add(self.tab2, text="Розгалужений")
self.notebook.add(self.tab3, text="Циклічний")

self.setup_tab1()
self.setup_tab2()
self.setup_tab3()

def show_image(self, parent, path, text):
    try:
        if not os.path.exists(path):
            raise Exception

        img = Image.open(path)
        w_base = 450
        ratio = (w_base / float(img.size[0]))
        h_new = int((float(img.size[1]) * float(ratio)))
        img = img.resize((w_base, h_new), Image.Resampling.LANCZOS)
        photo = ImageTk.PhotoImage(img)

        lbl = tk.Label(parent, image=photo)
        lbl.image = photo
        lbl.pack(pady=10)
    except:
        lbl = tk.Label(parent, text=text, font=("Cambria Math", 11), pady=15)
        lbl.pack()

def setup_tab1(self):
    self.show_image(self.tab1, CalculatorLogic.LINEAR_IMG, CalculatorLogic.LINEAR_FORMULA)

    frame = tk.LabelFrame(self.tab1, text="Вхідні дані")
    frame.pack(padx=20, pady=5, fill="x")

```

```
tk.Label(frame, text="a = ").grid(row=0, column=0, padx=5, pady=5)
self.entry_a = tk.Entry(frame)
self.entry_a.grid(row=0, column=1, padx=5, pady=5)
```

```
tk.Label(frame, text="b = ").grid(row=1, column=0, padx=5, pady=5)
self.entry_b = tk.Entry(frame)
self.entry_b.grid(row=1, column=1, padx=5, pady=5)
```

```
tk.Label(frame, text="c = ").grid(row=2, column=0, padx=5, pady=5)
self.entry_c = tk.Entry(frame)
self.entry_c.grid(row=2, column=1, padx=5, pady=5)
```

```
tk.Button(self.tab1, text="Обчислити", command=self.calc1).pack(pady=5)
self.res1 = tk.Label(self.tab1, text="Результат: -")
self.res1.pack(pady=10)
```

```
def calc1(self):
    try:
        a = float(self.entry_a.get())
        b = float(self.entry_b.get())
        c = float(self.entry_c.get())
        res = CalculatorLogic.calculate_linear(a, b, c)
        self.res1.config(text="Результат: " + str(round(res, 2)), fg="green")
    except Exception as e:
        self.res1.config(text="Помилка", fg="red")
        messagebox.showerror("Помилка", str(e))
```

```
def setup_tab2(self):
    self.show_image(self.tab2, CalculatorLogic.BRANCHING_IMG, CalculatorLogic.BRANCHING_FORMULA)
```

```
frame = tk.LabelFrame(self.tab2, text="Вхідні дані")
frame.pack(padx=20, pady=5, fill="x")
```

```
tk.Label(frame, text="k = ").grid(row=0, column=0, padx=5, pady=5)
self.entry_k = tk.Entry(frame)
self.entry_k.grid(row=0, column=1, padx=5, pady=5)
```

```
tk.Label(frame, text="d = ").grid(row=1, column=0, padx=5, pady=5)
self.entry_d = tk.Entry(frame)
self.entry_d.grid(row=1, column=1, padx=5, pady=5)
```

```
tk.Button(self.tab2, text="Обчислити", command=self.calc2).pack(pady=5)
self.res2 = tk.Label(self.tab2, text="Результат: -")
self.res2.pack(pady=10)
```

```
def calc2(self):
    try:
```

```

        k = float(self.entry_k.get())
        d = float(self.entry_d.get())
        res = CalculatorLogic.calculate_branching(k, d)
        self.res2.config(text="Результат: " + str(round(res, 2)), fg="green")
    except Exception as e:
        self.res2.config(text="Помилка", fg="red")
        messagebox.showerror("Помилка", str(e))

def setup_tab3(self):
    self.show_image(self.tab3, CalculatorLogic.CYCLIC_IMG, CalculatorLogic.CYCLIC_FORMULA)

    frame = tk.LabelFrame(self.tab3, text="Вхідні дані")
    frame.pack(padx=20, pady=5, fill="x")

    tk.Label(frame, text="n = ").grid(row=0, column=0, padx=5, pady=5)
    self.entry_n = tk.Entry(frame, width=10)
    self.entry_n.grid(row=0, column=1, padx=5, pady=5, sticky="w")

    tk.Button(frame, text="Заповнити випадковими значеннями", command=self.do_random).grid(row=0, column=2, padx=10)

    tk.Label(frame, text="A:").grid(row=1, column=0, padx=5, pady=5)
    self.text_a = tk.Text(frame, height=3, width=40)
    self.text_a.grid(row=1, column=1, columnspan=2, padx=5, pady=5)

    tk.Label(frame, text="B:").grid(row=2, column=0, padx=5, pady=5)
    self.text_b = tk.Text(frame, height=3, width=40)
    self.text_b.grid(row=2, column=1, columnspan=2, padx=5, pady=5)

    tk.Button(self.tab3, text="Обчислити", command=self.calc3).pack(pady=5)
    self.res3 = tk.Label(self.tab3, text="Результат: -")
    self.res3.pack(pady=5)

def do_random(self):
    try:
        n = int(self.entry_n.get())
        if n <= 0: return

        arr1 = []
        arr2 = []
        for i in range(n+1):
            arr1.append(random.randint(1, 20))
            arr2.append(random.randint(1, 20))

        self.text_a.delete("1.0", tk.END)
        self.text_b.delete("1.0", tk.END)

        s1 = ""
        for x in arr1:

```

```

s1 += str(x) + ", "

s2 = ""
for x in arr2:
    s2 += str(x) + ", "

self.text_a.insert("1.0", s1[:-2])
self.text_b.insert("1.0", s2[:-2])
except:
    messagebox.showwarning("Увага", "Введіть n")

def calc3(self):
    try:
        n = int(self.entry_n.get())
        str_a = self.text_a.get("1.0", tk.END)
        str_b = self.text_b.get("1.0", tk.END)

        list_a = []
        for x in str_a.replace("\n", " ").split(" "):
            if x.strip() != "":
                list_a.append(float(x))

        list_b = []
        for x in str_b.replace("\n", " ").split(" "):
            if x.strip() != "":
                list_b.append(float(x))

        res = CalculatorLogic.calculate_cyclic(n, list_a, list_b)
        self.res3.config(text="Результат: " + str(round(res, 2)), fg="green")
    except Exception as e:
        self.res3.config(text="Помилка", fg="red")
        messagebox.showerror("Помилка", str(e))

def load_file(self):
    idx = self.notebook.index(self.notebook.select())
    filename = filedialog.askopenfilename(filetypes=[("Text", "*.txt")])
    if not filename:
        return

    try:
        with open(filename, "r") as f:
            lines = f.readlines()

        clean_lines = []
        for l in lines:
            if l.strip() != "":
                clean_lines.append(l.strip())

```



```

if idx == 0:
    if len(clean_lines) < 3: return
    self.entry_a.delete(0, tk.END)
    self.entry_a.insert(0, clean_lines[0])
    self.entry_b.delete(0, tk.END)
    self.entry_b.insert(0, clean_lines[1])
    self.entry_c.delete(0, tk.END)
    self.entry_c.insert(0, clean_lines[2])

elif idx == 1:
    if len(clean_lines) < 2: return
    self.entry_k.delete(0, tk.END)
    self.entry_k.insert(0, clean_lines[0])
    self.entry_d.delete(0, tk.END)
    self.entry_d.insert(0, clean_lines[1])

elif idx == 2:
    if len(clean_lines) < 3: return
    self.entry_n.delete(0, tk.END)
    self.entry_n.insert(0, clean_lines[0])
    self.text_a.delete("1.0", tk.END)
    self.text_a.insert("1.0", clean_lines[1])
    self.text_b.delete("1.0", tk.END)
    self.text_b.insert("1.0", clean_lines[2])

messagebox.showinfo("Ок", "Завантажено")
except:
    messagebox.showerror("Помилка", "Не вдалося відкрити файл")

def save_file(self):
    idx = self.notebook.index(self.notebook.select())
    filename = filedialog.asksaveasfilename(defaultextension=".txt",
                                           filetypes=[("Text", "*.txt")])
    if not filename:
        return

    try:
        with open(filename, "w") as f:
            if idx == 0:
                f.write(self.entry_a.get() + "\n")
                f.write(self.entry_b.get() + "\n")
                f.write(self.entry_c.get() + "\n")

            elif idx == 1:
                f.write(self.entry_k.get() + "\n")
                f.write(self.entry_d.get() + "\n")

            elif idx == 2:

```

```

        f.write(self.entry_n.get() + "\n")
        f.write(self.text_a.get("1.0", tk.END).strip() + "\n")
        f.write(self.text_b.get("1.0", tk.END).strip() + "\n")

        messagebox.showinfo("Ок", "Файл збережено")
    except:
        messagebox.showerror("Помилка", "Не вдалося зберегти файл")

if __name__ == "__main__":
    app = MainWindow()
    app.mainloop()

```

Вміст файл MathLogic.py:

```

import math
class CalculatorLogic:
    LINEAR_FORMULA = "f =  $\sqrt[3]{5 + c \cdot \sqrt{b + 5\sqrt{a}}}$ "
    BRANCHING_FORMULA = (
        "k > 10 → f =  $\sqrt{k \cdot \sqrt{d^2} + d \cdot \sqrt{k^2}}$  \n"
        "k ≤ 10 → f = (k + d)^2"
    )
    CYCLIC_FORMULA = ("f =  $\prod_{i=1}^n (a_i + b_{i+1}) + \sum_i^n (a_{j+1} \cdot b_j)$ ")
    LINEAR_IMG = "media/linear.png"
    BRANCHING_IMG = "media/branching.png"
    CYCLIC_IMG = "media/cyclic.png"

    def calculate_linear(a, b, c):
        if a < 0:
            raise ValueError("Параметр 'a' повинен бути >= 0")
        term_b = b + 5 * math.sqrt(a)
        if term_b < 0:
            raise ValueError("Вираз (b + 5*sqrt(a)) повинен бути >= 0")
        base = 5 + c * math.sqrt(term_b)
        if base >= 0:
            return base ** (1 / 3)
        else:
            return - (abs(base) ** (1 / 3))

    def calculate_branching(k, d):
        if k > 10:
            val = k * abs(d) + d * k
            if val < 0:
                raise ValueError("Підкореневий вираз від'ємний")
            return math.sqrt(val)
        else:
            return (k + d) ** 2

    def calculate_cyclic(n, a_list, b_list):
        if len(a_list) < n + 1 or len(b_list) < n + 1:
            raise ValueError(f"Для n={n} потрібно {n+1} чисел у масивах!")

```

```

product_part = 1.0
sum_part = 0.0
for i in range(1, n + 1):
    product_part *= (a_list[i - 1] + b_list[i])
    sum_part += (a_list[i] * b_list[i - 1])
return product_part + sum_part

```

Результати виконання програми:

ЛР1

Файл

Лінійний Розгалужений Циклічний

$$Y1 = \sqrt[3]{5 + c \cdot \sqrt{b + 5\sqrt{a}}}$$

Вхідні дані:

a = 3

b = 4

c = 3

Обчислити

Результат: 2.84

Рисунок 4 – Результат обчислення лінійного алгоритму

ЛР1

Файл

Лінійний Розгалужений Циклічний

Обчислити, якщо $k > 10$

$$y = \sqrt{k \cdot \sqrt{d^2} + d \cdot \sqrt{k^2}}$$

інакше $y = (k + d)^2$

Вхідні дані:

k = 10

d = 3

Обчислити

Результат: 225.0

Рисунок 5 – Результат обчислення алгоритму, що розгалужується

The screenshot shows a software application window titled "ЛР1" with a menu bar containing "Файл". Below the menu bar are three tabs: "Лінійний", "Розгалужений", and "Циклічний", with the "Циклічний" tab selected. The main area displays the mathematical formula:

$$f = \prod_{i=1}^n (a_i + b_{i+1}) + \sum_{i=1}^n (a_{i+1} \cdot b_i)$$

Below the formula, there is a section titled "Вхідні дані" (Input data) containing the following elements:

- A label "n =" followed by a text input field containing the value "3". To the right of this field is a button labeled "Заповнити випадковими значеннями" (Fill with random values).
- A label "A:" followed by a text input field containing the values "1, 2, 3, 4".
- A label "B:" followed by a text input field containing the values "5, 6, 7, 8".

At the bottom of the input section is a button labeled "Обчислити" (Calculate). Below this button, the result is displayed in green text: "Результат: 749.0".

Рисунок 6 – Результат обчислення циклічного алгоритму

Висновки:

Отже, під час виконання лабораторної роботи було вивчено принципи побудови блок-схем лінійного, розгалуженого та циклічного алгоритмів. У ході роботи закріплено навички аналізу структури алгоритмів і послідовності їх виконання, а також створено програму згідно з варіантом. Програма працює коректно, помилок під час тестування і створення не було.